

Arsitektur & Panduan Migrasi: Deploy Mandiri ke VPS & Integrasi Google OAuth

Dokumen ini adalah rancangan arsitektur teknis dan panduan operasional untuk memindahkan deployment aplikasi dari ekosistem Hercules.ai ke VPS Mandiri (Self-Hosted) menggunakan Google OAuth / Standar OpenID Connect (OIDC) tanpa mengubah logika fitur, UI/UX, maupun model bisnis inti pada codebase.

1. Spesifikasi Server (Hardware & System Requirements)

Mengingat arsitektur backend Convex mengelola komputasi reaktif, caching in-memory, pengindeksan dokumen, dan WebSocket persisten, berikut adalah rekomendasi spesifikasi VPS:

A. Tabel Spesifikasi Hardware

Komponen	Spesifikasi Minimal (<i>Staging / Internal / <50 Users</i>)	Spesifikasi Rekomendasi (<i>Production Enterprise / 100–1000+ Users</i>)
CPU / vCPU	2 vCPU (x86_64 / ARM64)	4–8 vCPU (High Performance Compute)
RAM / Memory	4 GB RAM (+ 2 GB Swap)	8 GB – 16 GB RAM (ECC Recommended)
Storage / Disk	40 GB NVMe SSD	100 GB – 250 GB+ NVMe SSD (High IOPS)
Network Bandwidth	1 Gbps port / Min. 1 TB Transfer/bulan	1 Gbps – 10 Gbps port / Unmetered Bandwidth
Operating System	Ubuntu 22.04 LTS / Debian 12	Ubuntu 24.04 LTS (x86_64 / ARM64)
Runtimes & Engine	Node.js 20+ LTS, Docker 26+, pnpm 9+	Docker Engine + Compose, Traefik v3, Caddy, PostgreSQL 16, Redis

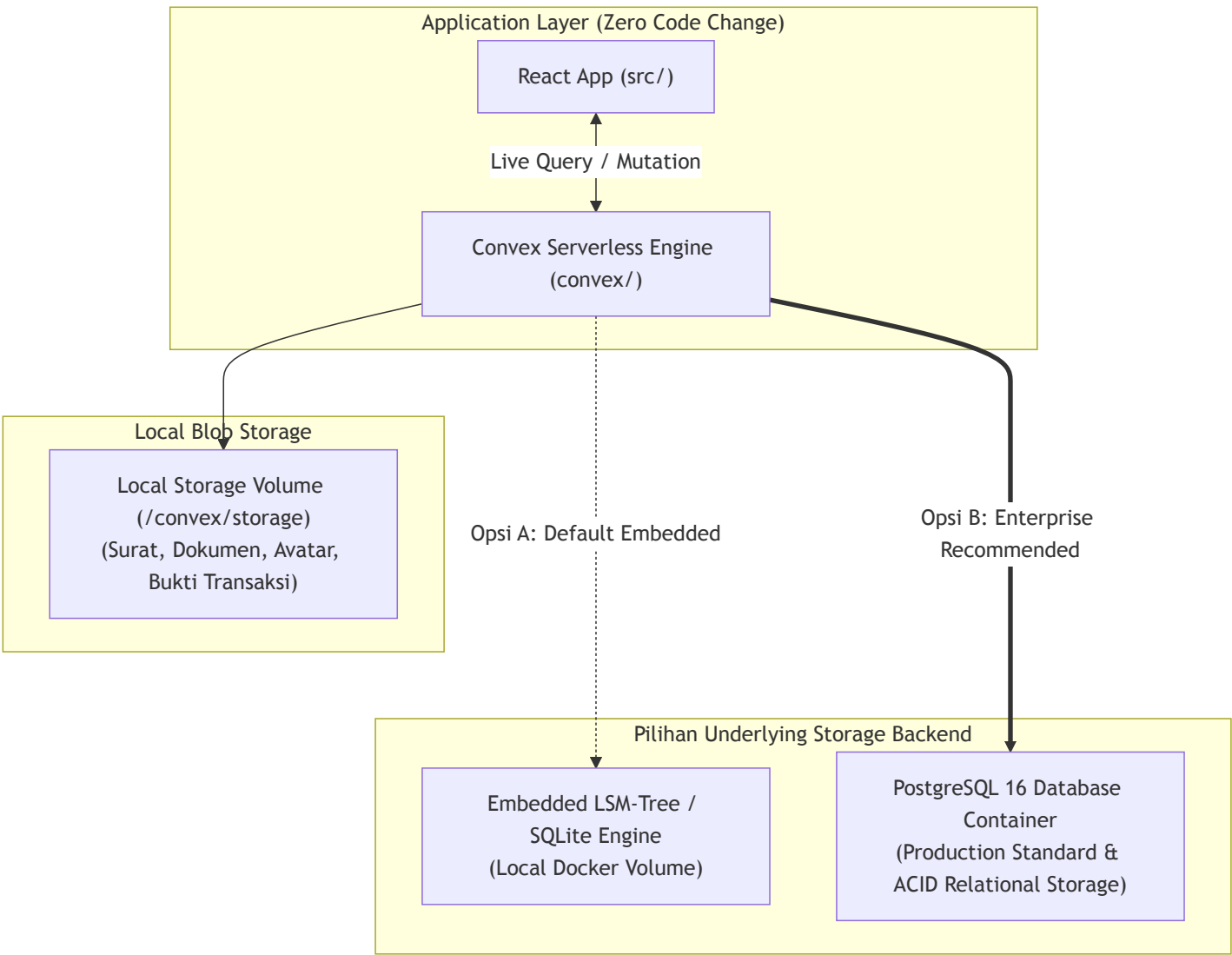
B. Rasionalisasi Beban Kerja Server:

- Frontend Static SPA (Caddy Container):** Sangat ringan, memory-safe (Go runtime), dan efisien (<30 MB RAM).
- Convex Backend + PostgreSQL Storage:** PostgreSQL menangani persistensi disk yang tangguh dan ACID-compliant, sedangkan Convex menangani komputasi reaktif in-memory. Alokasi 8–16 GB RAM memastikan query real-time dan indexing 249 tabel berjalan instan.

3. **Storage NVMe SSD:** Dibutuhkan IOPS tinggi untuk melayani read/write transaksi database PostgreSQL serta file upload lokal di filesystem VPS.

2. Analisis & Perbandingan Storage Engine Convex (Underlying Storage Strategy)

Di dalam Self-Hosted Convex Backend, kita memiliki dua strategi untuk lapisan penyimpanan bawah (*underlying persistence engine*):



A. Tabel Komparasi: SQLite/LSM Bawaan vs PostgreSQL Backend

Parameter Evaluasi	Ops A: SQLite / LSM Engine Bawaan	Ops B: PostgreSQL 16 Backend (Direkomendasikan)
Perubahan Kode Aplikasi	● Nol (Zero Code Change)	● Nol (Zero Code Change) – Kode Convex & React 100% tetap identik.

Parameter Evaluasi	Opsi A: SQLite / LSM Engine Bawaan	Opsi B: PostgreSQL 16 Backend (Direkomendasikan)
Konsumsi Resource (RAM/CPU)	🟢 Sangat rendah (hemat memori).	🟡 Menengah (membutuhkan tambahan alokasi ~500MB–1GB RAM untuk PostgreSQL).
Kemudahan Backup & Recovery	🟡 Terbatas pada snapshot folder volume binary.	🟢 Sangat Unggul – Mendukung <code>pg_dump</code> , automated daily backups, & WAL archiving.
Skalabilitas Data Volume	🟡 Optimal untuk database skala kecil–menengah (<10GB).	🟢 Sangat Tinggi – Teruji menangani database ratusan Gigabyte & jutaan records.
Tooling & Ekosistem SysAdmin	🔴 Minim tooling manajemen database eksternal.	🟢 Standar Industri – Kompatibel dengan pgAdmin, DBeaver, Prometheus Postgres Exporter.
Integritas & Crash Recovery	🟡 Bergantung pada snapshot binary LSM engine.	🟢 ACID Penuh – Sangat tahan terhadap insiden mati listrik / crash container tak terduga.

B. Pro & Cons Detail:

1. Opsi A: SQLite / LSM–Tree Bawaan (Default Embedded)

- **Pros:**
 - Konfigurasi Docker Compose sangat ringkas (tanpa container database terpisah).
 - Kecepatan baca/tulis lokal langsung in–process memory.
 - Cocok untuk server berkapasitas minimal (2 vCPU / 4 GB RAM) atau kebutuhan staging internal.
- **Cons:**
 - Kurang fleksibel untuk manajemen backup terpisah (disaster recovery lebih rumit).
 - Risiko korupsi data lebih tinggi jika terjadi *hard reboot* pada server saat penulisan intensif.

2. Opsi B: PostgreSQL 16 Storage Backend (Pilihan Terbaik Enterprise)

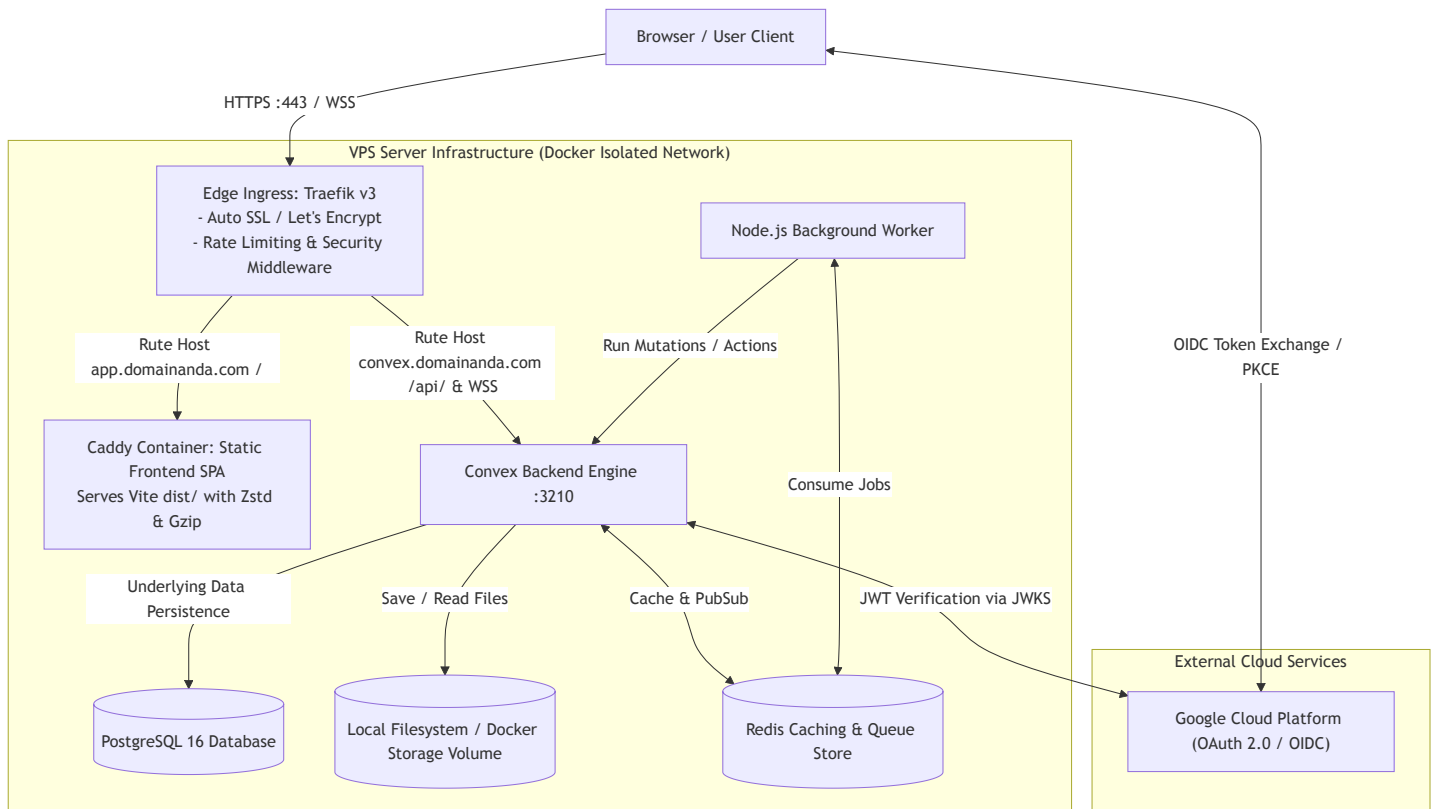
- **Pros:**
 - **Kestabilan Teruji:** Data 249 tabel disimpan di relational database paling andal di dunia.
 - **Standard Enterprise Backup:** Mudah di–backup setiap malam via script `pg_dump` ke storage cadangan.
 - **Zero Impact ke Kode:** Developer tetap menulis fungsi Convex biasa (`ctx.db.query()` , `ctx.db.insert()`) tanpa perlu tahu bahwa storage di belakangnya adalah PostgreSQL.

- **Cons:**
 - Memerlukan konfigurasi container PostgreSQL tambahan pada `docker-compose.yml` .

3. Tech Stack Infrastruktur & Layanan Server (Production-Ready)

Lapisan (Layer)	Komponen / Teknologi	Fungsi & Peran dalam Sistem
Edge Gateway & Ingress	Traefik v3	• Reverse proxy cloud-native utama, SSL/TLS Termination otomatis (Let's Encrypt ACME). • Dynamic service discovery via Docker labels, Rate Limiting, DDoS shielding, and WebSocket routing.
Frontend Static Web Server	Caddy (Alpine Container)	• Melayani file static hasil build Vite (<code>dist/</code>) dengan kompresi modern (Zstandard / Gzip). • Menangani SPA client-side routing fallback (<code>try_files {path} /index.html</code>) secara instan dan memory-safe.
Database Engine & Reactive Core	Convex Backend + PostgreSQL 16	• Real-time reactive document engine, ACID transactional mutations, automatic live subscriptions. • PostgreSQL 16 bertindak sebagai underlying persistent storage.
Caching & Pub/Sub	Redis 7 (Alpine)	Cache session, background job queue state, transient locks, and rate limit tracking.
Async Background Workers	Node.js / BullMQ Worker	Pemrosesan asynchronous job (email blast, invoice/pdf generation, data aggregation, sync analytics snapshot).
File & Attachment Storage	Local Volume Storage (Docker Volume / Local Disk)	Penyimpanan file lokal mandiri pada filesystem VPS untuk lampiran surat, avatar user, dokumen pegawai, dan bukti transaksi expense tanpa dependensi S3 eksternal.
Container Orchestration	Docker & Docker Compose	Manajemen lifecycle kontainer, isolated internal bridge networking, and health checks.
Identity Provider (IdP)	Google Cloud OAuth 2.0 (OIDC)	Otentikasi single sign-on standar industri menggantikan dependensi Hercules IdP.

4. Topologi Arsitektur VPS Mandiri (Traefik v3 + Caddy + Postgres Stack)



Keterangan Alur & Komponen Topologi:

1. Pintu Gerbang Utama (Traefik v3 Edge Ingress):

- Menjadi satu-satunya titik masuk publik yang membuka port **80 (HTTP)** dan **443 (HTTPS/WSS)**.
- Menangani otomatisasi penerbitan dan pembaruan sertifikat SSL Let's Encrypt, auto-redirect HTTP ke HTTPS, serta menyaring trafik mencurigakan via rate-limiting middleware sebelum diteruskan ke container internal.

2. Frontend Static SPA (Caddy Container):

- Menerima traffic untuk domain utama aplikasi (`app.domainanda.com`).
- Melayani aset statis hasil kompilasi Vite (`dist/`) dengan kompresi ultra-cepat (*Zstandard & Gzip*).
- Menjalankan fallback *client-side routing* SPA (`try_files {path} /index.html`) agar navigasi React Router DOM berjalan mulus tanpa error 404.

3. Backend Engine & WebSocket (Convex Backend):

- Menerima request API dan koneksi WebSocket persisten (`wss://`) untuk domain backend (`convex.domainanda.com`).

- Menjalankan deterministik *queries*, transaksi ACID *mutations*, dan *actions* untuk kalkulasi data 249 tabel secara reaktif.

4. Lapisan Penyimpanan Data (PostgreSQL 16 & Local File Storage):

- **PostgreSQL 16:** Menyimpan seluruh data tabel, indeks dokumen, dan histori mutasi di dalam database relasional yang stabil dan mudah di-backup via `pg_dump`.
- **Local File Storage Volume (`convex_storage`):** Menyimpan file binary fisik (PDF surat dinas, lampiran berkas, foto profil, dan kuitansi expense) langsung di filesystem VPS lokal.

5. Cache & Background Processing (Redis & Async Worker):

- **Redis 7:** Berfungsi sebagai in-memory message broker, penahan state queue antrean, dan lock terdistribusi.
- **Node.js Worker:** Mengonsumsi task asynchronous berat dari Redis (seperti agregasi snapshot analitik, pembuatan dokumen PDF massal, dan pengiriman notifikasi/email) agar performa API utama tetap responsif.

6. Otentikasi Terdistribusi (Google OAuth / OIDC):

- Klien (browser) melakukan otentikasi login langsung ke Google Identity Platform menggunakan alur PKCE.
- Token JWT yang diterima diteruskan ke Convex Backend, dan backend memverifikasi tanda tangan kriptografi token secara independen melalui Google JWKS (*JSON Web Key Set*).

5. Analisis Titik Penyelarasan Codebase (Migration Touchpoints)

Sesuai aturan kerja, modifikasi codebase **dibatasi secara ketat** hanya pada layer adapter otentikasi dan environment konfigurasi:

Komponen	Implementasi Saat Ini (Hercules.ai)	Penyelarasan Menuju VPS & Google OAuth
Backend Auth Provider	<code>convex/auth.config.js</code> (Hercules OIDC Authority)	Menggunakan Google OIDC Authority (<code>https://accounts.google.com</code>)
Frontend Auth Provider	<code>src/components/providers/auth.tsx</code> (<code>@usehercules/auth</code>)	Menggunakan standard <code>react-oidc-context</code> atau adapter OIDC native
Convex Client Auth Adapter	<code>src/components/providers/convex.tsx</code> (<code>ConvexProviderWithHerculesAuth</code>)	Menggunakan standard <code>ConvexProviderWithAuth</code> dari <code>convex/react</code>

Komponen	Implementasi Saat Ini (Hercules.ai)	Penyelarasan Menuju VPS & Google OAuth
Auth Callback	<code>src/pages/auth/Callback.tsx</code> (<code>useAuthCallback</code>)	Menggunakan standard OIDC callback handler
Auth Hooks	<code>src/hooks/use-auth.ts</code> (<code>@usehercules/auth/react</code>)	Wrapper hook yang mengembalikan format user/auth identik

6. Konfigurasi Deployment Multi-Service (`docker-compose.yml` & `Caddyfile`)

A. File Konfigurasi Frontend (`Caddyfile`)

```
:80 {
  root * /usr/share/caddy
  encode zstd gzip
  file_server
  try_files {path} /index.html
}
```

caddy

B. File Orchestration (`docker-compose.yml`)

```
version: '3.8'

services:
  # 1. Edge Gateway & Auto-SSL (Traefik v3)
  traefik:
    image: traefik:v3.1
    command:
      - "--api.insecure=false"
      - "--providers.docker=true"
      - "--providers.docker.exposedbydefault=false"
      - "--entryPoints.web.address=:80"
      - "--entryPoints.web.http.redirects.entryPoint.to=websecure"
      - "--entryPoints.web.http.redirects.entryPoint.scheme=https"
      - "--entryPoints.websecure.address=:443"
      - "--certificatesresolvers.myresolver.acme.tlschallenge=true"
      - "--certificatesresolvers.myresolver.acme.email=admin@domainanda.com"
      - "--certificatesresolvers.myresolver.acme.storage=/letsencrypt/acme.json"
    ports:
```

yaml

```
- "80:80"
- "443:443"
volumes:
- /var/run/docker.sock:/var/run/docker.sock:ro # Read-only for security
- ./letsencrypt:/letsencrypt
networks:
- app-network
restart: always
```

2. Frontend Static SPA Server (Caddy)

```
frontend:
  image: caddy:2-alpine
  volumes:
    - ./dist:/usr/share/caddy:ro
    - ./Caddyfile:/etc/caddy/Caddyfile:ro
  labels:
    - "traefik.enable=true"
    - "traefik.http.routers.frontend.rule=Host(`app.domainanda.com`)"
    - "traefik.http.routers.frontend.entrypoints=websecure"
    - "traefik.http.routers.frontend.tls.certresolver=myresolver"
    - "traefik.http.services.frontend.loadbalancer.server.port=80"
  networks:
    - app-network
  restart: always
```

3. Persistent Database Engine (PostgreSQL 16)

```
postgres-db:
  image: postgres:16-alpine
  environment:
    - POSTGRES_USER=convex
    - POSTGRES_PASSWORD=${POSTGRES_PASSWORD}
    - POSTGRES_DB=convex_database
  volumes:
    - pg_data:/var/lib/postgresql/data
  networks:
    - app-network
  restart: always
```

4. Reactive Database & Backend Engine (Convex)

```
convex-backend:
  image: ghcr.io/get-convex/convex-backend:latest
  environment:
    - PORT=3210
    - CONVEX_CLOUD_ORIGIN=https://convex.domainanda.com
    - CONVEX_SITE_ORIGIN=https://app.domainanda.com
    - GOOGLE_CLIENT_ID=${GOOGLE_CLIENT_ID}
    - DATABASE_URL=postgresql://convex:${POSTGRES_PASSWORD}@postgres-
db:5432/convex_database
  volumes:
    - convex_storage:/convex/storage # Local file storage volume
  labels:
```

```
- "traefik.enable=true"
- "traefik.http.routers.convex.rule=Host(`convex.domainanda.com`)"
- "traefik.http.routers.convex.entrypoints=websecure"
- "traefik.http.routers.convex.tls.certresolver=myresolver"
- "traefik.http.services.convex.loadbalancer.server.port=3210"
```

depends_on:

```
- postgres-db
```

networks:

```
- app-network
```

restart: always

5. In-Memory Cache & Queue Broker (Redis)

redis:

image: redis:7-alpine

command: redis-server --appendonly yes --requirepass \${REDIS_PASSWORD}

volumes:

```
- redis_data:/data
```

networks:

```
- app-network
```

restart: always

6. Async Worker (Heavy Calculations / PDF / Notifications)

app-worker:

build:

context: .

dockerfile: Dockerfile.worker

environment:

```
- REDIS_URL=redis://:${REDIS_PASSWORD}@redis:6379
```

```
- CONVEX_URL=http://convex-backend:3210
```

depends_on:

```
- redis
```

```
- convex-backend
```

networks:

```
- app-network
```

restart: always

networks:

app-network:

driver: bridge

volumes:

pg_data:

convex_storage:

redis_data:

7. Spesifikasi Konfigurasi Google OAuth

A. Konfigurasi Google Cloud Console

- 1. **Application Type:** Web Application
- 2. **Authorized JavaScript Origins:**
 - `https://app.domainanda.com`
 - `http://localhost:5173` (Local Dev)
- 3. **Authorized Redirect URIs:**
 - `https://app.domainanda.com/auth/callback`
 - `http://localhost:5173/auth/callback`

B. Konfigurasi Backend (`convex/auth.config.js`)

```
export default {
  providers: [
    {
      domain: "https://accounts.google.com",
      applicationID: process.env.GOOGLE_CLIENT_ID,
    },
  ],
};
```

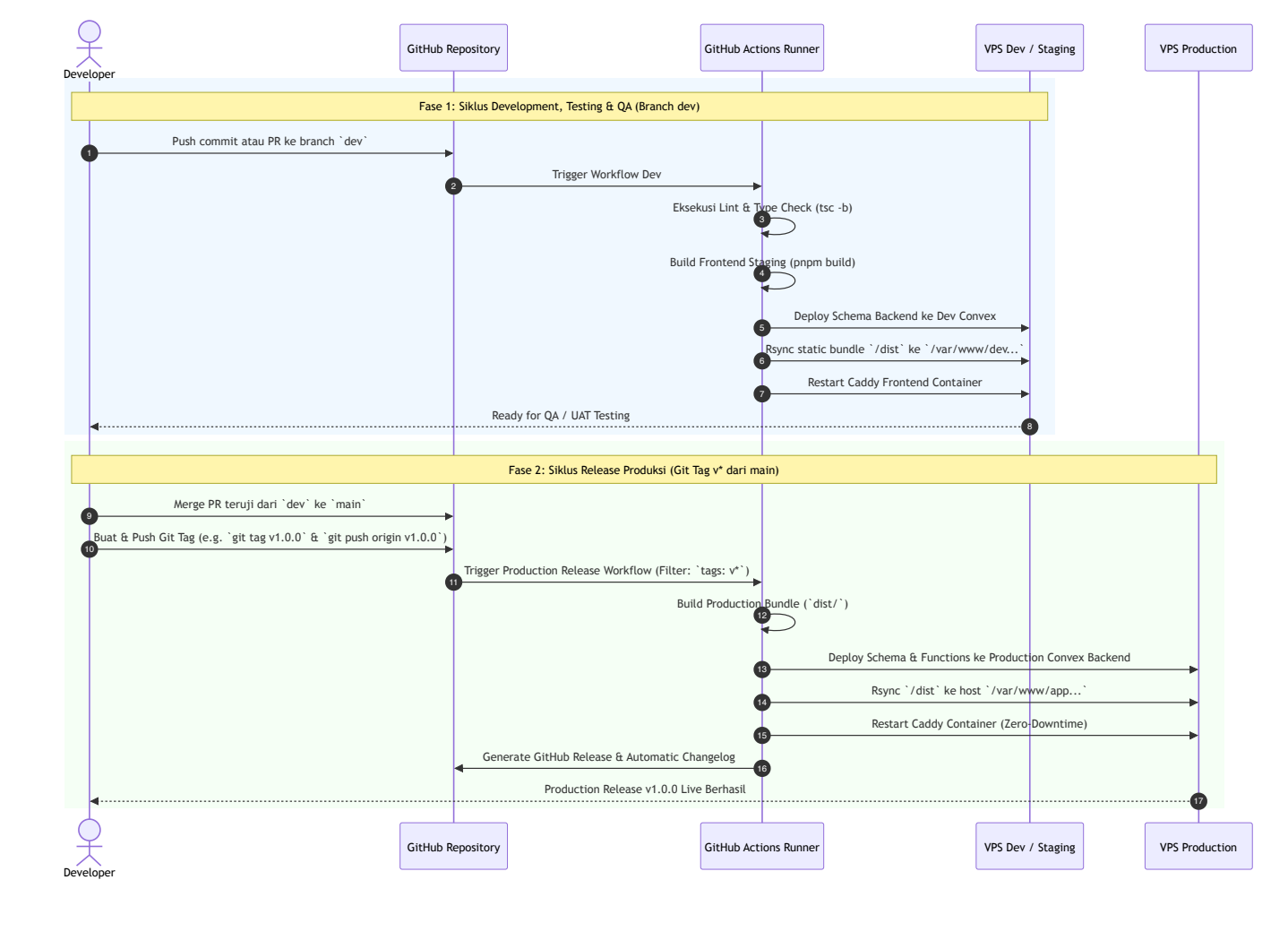
javascript

8. Matriks Environment Variables VPS

Variable	Lingkungan	Keterangan
VITE_CONVEX_URL	Frontend (<code>.env.production</code>)	Endpoint Convex (<code>https://convex.domainanda.com</code> / Self-Hosted)
VITE_OIDC_AUTHORITY	Frontend (<code>.env.production</code>)	<code>https://accounts.google.com</code>
VITE_OIDC_CLIENT_ID	Frontend (<code>.env.production</code>)	Client ID Google Cloud Console
VITE_OIDC_REDIRECT_URI	Frontend (<code>.env.production</code>)	<code>https://app.domainanda.com/auth/callback</code>
GOOGLE_CLIENT_ID	Backend Convex Env	Client ID Google Cloud Console untuk verifikasi JWT

Variable	Lingkungan	Keterangan
POSTGRES_PASSWORD	Database & Backend Env	Password master database PostgreSQL
REDIS_PASSWORD	Backend & Worker Env	Password akses Redis server

9. Prosedur CI/CD Deployment Otomatis



A. Workflow 1: Development & Testing (`.github/workflows/dev-deploy.yml`)

- **Trigger:** Setiap `push` ke branch `dev` atau Pull Request ke `dev`.
- **Tujuan:** Memastikan integritas kode (lint, typecheck, build) dan memperbarui server development secara otomatis untuk testing.

name: Dev CI/CD Pipeline

yaml

```
on:
  push:
    branches:
      - dev
  pull_request:
    branches:
      - dev

jobs:
  test-and-build:
    name: Lint, Test & Build
    runs-on: ubuntu-latest
    steps:
      - name: Checkout Code
        uses: actions/checkout@v4

      - name: Setup Node.js & pnpm
        uses: pnpm/action-setup@v3
        with:
          version: 9

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: 'pnpm'

      - name: Install Dependencies
        run: pnpm install --frozen-lockfile

      - name: Run Linter & Type Check
        run: |
          pnpm lint
          pnpm exec tsc -b

      - name: Build Frontend (Staging / Dev)
        env:
          VITE_CONVEX_URL: ${ secrets.DEV_CONVEX_URL }
          VITE_OIDC_AUTHORITY: https://accounts.google.com
          VITE_OIDC_CLIENT_ID: ${ secrets.DEV_GOOGLE_CLIENT_ID }
          VITE_OIDC_REDIRECT_URI: https://dev.domainanda.com/auth/callback
        run: pnpm build

  deploy-dev:
    name: Deploy to Dev VPS
    needs: test-and-build
    if: github.ref == 'refs/heads/dev' && github.event_name == 'push'
    runs-on: ubuntu-latest
    steps:
      - name: Checkout Code
        uses: actions/checkout@v4
```

- name: Setup Node.js & pnpm
uses: pnpm/action-setup@v3
with:
 version: 9
- name: Install Dependencies & Build
env:
 VITE_CONVEX_URL: \${{ secrets.DEV_CONVEX_URL }}
 VITE_OIDC_AUTHORITY: https://accounts.google.com
 VITE_OIDC_CLIENT_ID: \${{ secrets.DEV_GOOGLE_CLIENT_ID }}
 VITE_OIDC_REDIRECT_URI: https://dev.domainanda.com/auth/callback
run: |
 pnpm install --frozen-lockfile
 pnpm build
- name: Deploy Backend Schema to Dev Convex
env:
 CONVEX_DEPLOY_KEY: \${{ secrets.DEV_CONVEX_DEPLOY_KEY }}
run: npx convex deploy
- name: Deploy Frontend Build to Dev VPS via SSH
uses: appleboy/scp-action@v0.1.7
with:
 host: \${{ secrets.DEV_VPS_HOST }}
 username: \${{ secrets.DEV_VPS_USER }}
 key: \${{ secrets.DEV_VPS_SSH_KEY }}
 source: "dist/*"
 target: "/var/www/dev.domainanda.com"
 strip_components: 1
- name: Restart Caddy Container on Dev VPS
uses: appleboy/ssh-action@v1.0.3
with:
 host: \${{ secrets.DEV_VPS_HOST }}
 username: \${{ secrets.DEV_VPS_USER }}
 key: \${{ secrets.DEV_VPS_SSH_KEY }}
 script: |
 cd /opt/app-stack && docker compose restart frontend

B. Workflow 2: Production Release ([.github/workflows/prod-release.yml](#))

- **Trigger:** Khusus pembuatan **Git Tag** dengan pola `v*` (contoh: `v1.0.0` , `v1.2.3-patch`).
- **Tujuan:** Membangun versi final production, melakukan sinkronisasi backend production, rilis zero-downtime, dan generate release note.

```
name: Production Release Pipeline

on:
  push:
    tags:
      - 'v*'

jobs:
  production-release:
    name: Build & Deploy Production
    runs-on: ubuntu-latest
    steps:
      - name: Checkout Release Tag
        uses: actions/checkout@v4

      - name: Setup pnpm
        uses: pnpm/action-setup@v3
        with:
          version: 9

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: 'pnpm'

      - name: Install Dependencies
        run: pnpm install --frozen-lockfile

      - name: Build Production Frontend
        env:
          VITE_CONVEX_URL: ${ secrets.PROD_CONVEX_URL }
          VITE_OIDC_AUTHORITY: https://accounts.google.com
          VITE_OIDC_CLIENT_ID: ${ secrets.PROD_GOOGLE_CLIENT_ID }
          VITE_OIDC_REDIRECT_URI: https://app.domainanda.com/auth/callback
        run: pnpm build

      - name: Deploy Production Convex Backend
        env:
          CONVEX_DEPLOY_KEY: ${ secrets.PROD_CONVEX_DEPLOY_KEY }
        run: npx convex deploy

      - name: Deploy Frontend Artifacts to Production VPS
        uses: appleboy/scp-action@v0.1.7
        with:
          host: ${ secrets.PROD_VPS_HOST }
          username: ${ secrets.PROD_VPS_USER }
          key: ${ secrets.PROD_VPS_SSH_KEY }
          source: "dist/*"
          target: "/var/www/app.domainanda.com"
```

```
strip_components: 1
```

- name: Execute Zero-Downtime Reload on Production VPS
uses: appleboy/ssh-action@v1.0.3
with:
 host: \${ secrets.PROD_VPS_HOST }
 username: \${ secrets.PROD_VPS_USER }
 key: \${ secrets.PROD_VPS_SSH_KEY }
 script: |
 cd /opt/app-stack && docker compose restart frontend
 echo "Production Release \${ github.ref_name } Deployed Successfully!"
- name: Create GitHub Release
uses: softprops/action-gh-release@v2
with:
 tag_name: \${ github.ref_name }
 name: Release \${ github.ref_name }
 generate_release_notes: true

10. Struktur Proyek Terbaru (Project Structure)

Struktur proyek mencerminkan codebase yang siap dideploy ke VPS mandiri dengan seluruh konfigurasi infrastruktur dan pipeline CI/CD:

```
start-app/  
├── .github/  
│   ├── workflows/  
│   │   ├── dev-deploy.yml      # CI/CD Pipeline Staging/Dev (Push branch dev)  
│   │   └── prod-release.yml    # CI/CD Release Pipeline (Trigger Git Tag v*)  
├── convex/  
│   ├── _generated/            # Auto-generated types (api.d.ts, dataModel.d.ts)  
│   ├── auth.config.js         # Konfigurasi OIDC Google Authority & Client ID  
│   ├── crons.ts               # Background schedulers & snapshot generation  
│   ├── schema.ts              # Definisi 249 tabel database & index  
│   └── *.ts                   # Endpoints query, mutation & action per modul  
bisnis  
├── docs/  
│   ├── ARCHITECTURE.md        # Dokumentasi Teknis Sistem  
│   └── VPS_DEPLOYMENT_ARCHITECTURE.md # Arsitektur modul bawaan Hercules.ai  
├── src/  
│   ├── components/            # Arsitektur deploy VPS mandiri & Google OAuth  
│   │   ├── providers/         # Frontend React 19 Application Layer  
│   │   │   ├── auth.tsx       # UI components, Radix primitives, Providers  
│   │   └── convex.tsx         # Standard OIDC Auth Provider  
│   ├── hooks/                 # Standard Convex Auth Provider  
│   └── use-auth.ts            # Auth Hook adapter
```


- **Tantangan:** Memastikan pembaruan schema database di Convex Backend telah aktif sebelum aset frontend terbaru di Caddy disajikan ke pengguna, guna menghindari *schema mismatch* pada sesi aktif.

B. Matriks Evaluasi Beban Kerja Operasional:

Area Implementasi & Operasional	Tingkat Kesulitan	Kompleksitas Teknis
Penyelarasan Kode Auth (Google OIDC)	● Rendah (Low)	Hanya menyelaraskan 5 file adapter tanpa menyentuh logika fitur.
Setup Docker Network & Multi-Container	● Rendah (Low)	Orkestrasi 6 service terpusat di <code>docker-compose.yml</code> .
Tuning Ingress Traefik (SSL & WebSocket)	● Menengah (Moderate)	Konfigurasi ACME auto-renew, timeout keepalive, & rate limiting.
Konfigurasi PostgreSQL & Memory Tuning	● Menengah (Moderate)	Pembatasan buffer memory, konfigurasi user, & persistence volume.
<i>Pipeline CI/CD Dual-Stage (dev & tag v)*</i>	● Menengah (Moderate)	Manajemen secret GitHub Actions, deploy key, & SSH sync.
Otomasi Backup, Retensi & Disaster Recovery	● Menengah (Moderate)	Script backup otomatis <code>pg_dump</code> & sinkronisasi local storage.
Stabilitas WebSocket & Load Handling	● Tinggi (High)	Uji ketahanan koneksi persisten dan tuning socket OS.